

Aula 2

Programação II

Prof. Elton Masaharu Sato

Conversa Inicial

- Como visto na aula anterior, o Flutter utiliza o Dart como linguagem de programação
- O foco desta aula será o Dart

- Nesta aula, iremos aprender mais sobre o Dart e algumas das características que o tornam único e ideal para o desenvolvimento de aplicativos para aparelhos móveis

Dart: o que é?

Objetivos

- Otimização para UI
 - Funções assíncronas e espera
 - ✓ Permite que operações lentas não atrapalhem o tempo de processamento da aplicação
 - ✓ Ex.: carregar imagens

- **Funções para trabalhar com coleções**
 - ✓ **Coleções são agrupamento de dados. São ótimas para trabalhar com dados vindo de servidores**
 - ✓ **Ex.: lista de produtos**

- **Linguagem de alto nível**
 - ✓ **Facilita a programação feita pelo desenvolvedor**

	<pre>class Segment { int links = 4; toString() => 'I have \$links links'; }</pre>
	<pre>class Segment { var links: Int = 4 override fun toString(): String { return "I have \$links links" } }</pre>
	<pre>class Segment: CustomStringConvertible { var links: Int = 4 public var description: String { return "I have \(\$links) links" } }</pre>
	<pre>class Segment { links: number = 4 public toString = () => string => { return "I have \${this.links} links" }; }</pre>

Fonte: dart.dev, 2022.

- **Ambiente de desenvolvimento produtivo**
 - **Hot reload**
 - ✓ **Uma das ferramentas mais potentes, permite que o usuário teste códigos sem precisar carregar o código do zero**

- **Ferramentas de análise de código**
 - **O Dart consegue apontar erros no código antes mesmo de o código ser propriamente compilado**
 - **Ex.: variável do tipo errado**

- **Ferramentas de análise de execução e debug**
 - **Suas ferramentas são integradas no Dart, portanto, funcionam independentemente do editor escolhido**

- Eficiência multiplataforma
- Compilação *ahead of time* (AOT)
 - ✓ Uma das formas que o código pode ser compilado, otimiza o aplicativo para ser publicado

- Compilação *just in time* (JIT)
 - Uma das formas que o código pode ser compilado, habilita ferramentas como o *Hot Reload* e o *Debugger*

- Dart Web
 - ✓ Permite que o Dart funcione em aplicações web via JavaScript

DartPad

- Compilador *on-line* para realizar pequenos testes de código
- *Open source*

Operadores e Variáveis

Operadores: aritméticos

- $X + Y$: adição de X mais Y
- $X - Y$: subtração de X menos Y
- $X * Y$: multiplicação de X vezes Y
- X / Y : divisão de X dividido por Y

- $X \sim / Y$: adição de X mais Y
- $X \% Y$: restante da divisão X dividido por Y
- $X++$: incremento, "X recebe X + 1"
- $X--$: decremento, "X recebe X - 1"

Operadores: relacionais

- $X > Y$: X é maior que Y
- $X < Y$: X é menor que Y
- $X \geq Y$: X é maior ou igual a Y
- $X \leq Y$: X é menor ou igual a Y
- $X == Y$: X é igual a Y
- $X != Y$: X é diferente de Y

	X = 10, Y = 20	X = 15, Y = 15	X = 20, Y = 10
$X > Y$	Falso	Falso	Verdadeiro
$X < Y$	Verdadeiro	Falso	Falso
$X \geq Y$	Falso	Verdadeiro	Verdadeiro
$X \leq Y$	Verdadeiro	Verdadeiro	Falso
$X == Y$	Falso	Verdadeiro	Falso
$X != Y$	Verdadeiro	Falso	Verdadeiro

Fonte: Sato, 2022.

Operadores: tipo

- $X \text{ is } T$: X é do tipo T
- $X \text{ is! } T$: X não é do tipo T

Operadores: lógica

- $\&\&$: operador AND, "e" lógico
- $||$: operador OR, "ou" lógico
- NOT : operador NOT, "não" lógico

Operadores: designadores

- $X = Y$: X recebe o valor de Y
- $X += Y$: X recebe a soma de X e Y
- $X -= Y$: X recebe a subtração de X por Y
- $X *= Y$: X recebe a multiplicação de X e Y
- $X /= Y$: X recebe a divisão de X por Y
- $X \% = Y$: X recebe o restante da divisão de X por Y

Variáveis

- ▀ **Var:** variável genérica
- ▀ **Int:** número inteiro
- ▀ **Bool :** valor booleano, verdadeiro ou falso
- ▀ **String:** sequência de caracteres

Variáveis: coleções

- ▀ **List:** lista ordenada e indexada
- ▀ **Set:** conjunto não ordenado, evita duplicatas
- ▀ **Map:** grupo não ordenado, mas mapeado por chaves

Classes

Membros de uma classe

- ▀ **Campos:** as variáveis de uma classe
- ▀ **Acessos:** funções de *set* e *get* que acessam as variáveis
- ▀ **Construtor:** método especial que permite criar um objeto desta classe
- ▀ **Métodos:** as funções da classe. Acessos e construtores são considerados métodos

```
1 //classe food
2 class Comida{
3
4     String nome;
5     int quantidade;
6
7     //construtor de comida
8     Comida(this.nome, this.quantidade);
9
10    //acessor set de quantidade
11    set setQuantidade(int novaQuantidade){
12        quantidade = novaQuantidade;
13    }
14
15    //acessor get de nome
16    String get nomeDaComida{
17        return nome;
18    }
19
20    //método que passa uma String com a quantidade e nome
21    String estoque(){
22        return "Quantidade x $nome";
23    }
24 }
25 }
```

Fonte: Sato, 2022.

```
27 //main
28 void main() {
29     //declara e constrói um objeto da classe comida
30     Comida comida1 = Comida("Hamburger", 4);
31
32     //imprime o retorno do método estoque do objeto comida1
33     print(comida1.estoque());
34
35     //usa o acessor set para mudar a quantidade de comida1
36     comida1.setQuantidade = 2;
37
38     //imprime na tela a situação do estoque de comida1
39     print("Estoque de " + comida1.nomeDaComida + " foram alterados.");
40
41     //imprime o retorno do método estoque do objeto comida1
42     print(comida1.estoque());
43 }
```

Fonte: Sato, 2022.

Console

```
4 x Hamburger  
Estoque de Hamburger foram alterados.  
2 x Hamburger
```

Fonte: Sato, 2022.

Método toString

- Um método opcional bastante comum em algumas linguagens de programação. Este método devolve uma *string* que identifica o objeto
- Recomendado para facilitar tarefas de *debug*

```
20 //método que passa uma String com a quantidade e nome  
21 String toString() => "$quantidade x $nome";  
22  
23 }
```

Fonte: Sato, 2022.

Abstratas e Interface

Classe abstrata

- Tem como objetivo ser uma classe genérica para ser utilizada por várias classes
- Só possui métodos vazios para serem preenchidos pelas classes que implementam a classe
- Não pode ser instanciada

Implements

- Uma classe pode implementar uma classe abstrata através do "*implements*"
- Uma classe que implementa uma classe abstrata deve implementar todos os seus métodos

```

1 void main() {
2   // Pessoa p1 = Boxeador();
3   Pessoa p2 = Pessoa();
4   p2.setForca = 10;
5   print('Força do Soco do Boxeador: ${p1.soco() }');
6 }
7
8 abstract class Pessoa {
9   int soco();
10  set setForca(int forca);
11 }
12
13 class Boxeador implements Pessoa {
14   int forca = 0;
15   Boxeador();
16   override
17   int soco() {
18     set setForca(int forca) {
19       this.forca = forca;
20     }
21   }
22   override
23   int soco() {
24     return forca;
25   }
26 }

```

Fonte: Sato, 2022.

```

Console
Error compiling to JavaScript:
Info: Compiling with sound null safety
Warning: Interpreting this as package URI, 'package:dartpad_sample/main.dart'.
lib/main.dart:43:15:
Error: The class 'Pessoa' is abstract and can't be instantiated.
  Pessoa p2 = Pessoa();
                  ^^^^^^
Error: Compilation failed.

```

Fonte: Sato, 2022.

Console

Força do Soco do Boxeador: 10

Fonte: Sato, 2022.

Extends

- Uma classe pode ser uma extensão de outra classe através do "extends"
- Uma classe que estende outra classe é considerada filha desta outra classe
- Uma extensão não precisa ter todos os métodos da classe que ela estende

```

37 class Heroi extends Boxeador {
38
39   @override
40   int soco() {
41     return 100 * forca;
42   }
43 }

```

Fonte: Sato, 2022.

Console

Força do Soco do Boxeador: 10
Força do Soco do Heroi: 1000

Fonte: Sato, 2022.

Assíncrono e Futuro

- **Permite que uma operação lenta rode em paralelo enquanto o código faz outras tarefas**
- **Ex.: baixar imagens da internet, conectar a um servidor, etc.**

```

1  import javax.swing.*;
2
3  public class ParametrosA {
4
5      //atributos
6      private String nome;
7      private String endereco;
8      private String telefone;
9      private String dataNascimento;
10
11      //construtor
12      public ParametrosA() {
13
14          //atributos
15          nome = "";
16          endereco = "";
17          telefone = "";
18          dataNascimento = "";
19
20      }
21
22      //getters
23      public String getNome() {
24          return nome;
25      }
26
27      public String getEndereco() {
28          return endereco;
29      }
30
31      public String getTelefone() {
32          return telefone;
33      }
34
35      public String getDataNascimento() {
36          return dataNascimento;
37      }
38
39      //setters
40      public void setNome(String nome) {
41          this.nome = nome;
42      }
43
44      public void setEndereco(String endereco) {
45          this.endereco = endereco;
46      }
47
48      public void setTelefone(String telefone) {
49          this.telefone = telefone;
50      }
51
52      public void setDataNascimento(String dataNascimento) {
53          this.dataNascimento = dataNascimento;
54      }
55
56      //metodo para exibir os dados
57      public void exibirDados() {
58          System.out.println("Nome: " + nome);
59          System.out.println("Endereco: " + endereco);
60          System.out.println("Telefone: " + telefone);
61          System.out.println("Data de Nascimento: " + dataNascimento);
62      }
63
64      //metodo para ler os dados
65      public void lerDados() {
66          nome = JOptionPane.getTextField("Nome");
67          endereco = JOptionPane.getTextField("Endereco");
68          telefone = JOptionPane.getTextField("Telefone");
69          dataNascimento = JOptionPane.getTextField("Data de Nascimento");
70      }
71
72      //metodo para validar os dados
73      public boolean validarDados() {
74          return nome.length() > 0 && endereco.length() > 0 && telefone.length() > 0 && dataNascimento.length() > 0;
75      }
76
77      //metodo para salvar os dados
78      public void salvarDados() {
79          //chamar o metodo lerDados()
80          lerDados();
81
82          //chamar o metodo validarDados()
83          boolean validado = validarDados();
84
85          //se validado for verdadeiro, chamar o metodo exibirDados()
86          if (validado) {
87              exibirDados();
88          }
89      }
90
91      //metodo para exibir a mensagem de boas vindas
92      public void exibirMensagem() {
93          JOptionPane.showMessageDialog(null, "Bem-vindo ao sistema de cadastro de pessoas!");
94      }
95
96      //metodo para exibir a mensagem de despedida
97      public void exibirMensagemDespedida() {
98          JOptionPane.showMessageDialog(null, "Obrigado por usar o sistema de cadastro de pessoas!");
99      }
100
101      //metodo para exibir a mensagem de erro
102      public void exibirMensagemErro() {
103          JOptionPane.showMessageDialog(null, "Erro ao salvar os dados!");
104      }
105
106      //metodo para exibir a mensagem de sucesso
107      public void exibirMensagemSucesso() {
108          JOptionPane.showMessageDialog(null, "Dados salvos com sucesso!");
109      }
110
111      //metodo para exibir a mensagem de confirmação
112      public void exibirMensagemConfirmacao() {
113          JOptionPane.showMessageDialog(null, "Confirmação de dados!");
114      }
115
116      //metodo para exibir a mensagem de cancelamento
117      public void exibirMensagemCancelamento() {
118          JOptionPane.showMessageDialog(null, "Cancelamento de dados!");
119      }
120
121      //metodo para exibir a mensagem de exclusão
122      public void exibirMensagemExclusao() {
123          JOptionPane.showMessageDialog(null, "Exclusão de dados!");
124      }
125
126      //metodo para exibir a mensagem de atualização
127      public void exibirMensagemAtualizacao() {
128          JOptionPane.showMessageDialog(null, "Atualização de dados!");
129      }
130
131      //metodo para exibir a mensagem de busca
132      public void exibirMensagemBusca() {
133          JOptionPane.showMessageDialog(null, "Busca de dados!");
134      }
135
136      //metodo para exibir a mensagem de listagem
137      public void exibirMensagemListagem() {
138          JOptionPane.showMessageDialog(null, "Listagem de dados!");
139      }
140
141      //metodo para exibir a mensagem de relatório
142      public void exibirMensagemRelatorio() {
143          JOptionPane.showMessageDialog(null, "Relatório de dados!");
144      }
145
146      //metodo para exibir a mensagem de gráfico
147      public void exibirMensagemGráfico() {
148          JOptionPane.showMessageDialog(null, "Gráfico de dados!");
149      }
150
151      //metodo para exibir a mensagem de tabela
152      public void exibirMensagemTabela() {
153          JOptionPane.showMessageDialog(null, "Tabela de dados!");
154      }
155
156      //metodo para exibir a mensagem de formulário
157      public void exibirMensagemFormulario() {
158          JOptionPane.showMessageDialog(null, "Formulário de dados!");
159      }
160
161      //metodo para exibir a mensagem de menu
162      public void exibirMensagemMenu() {
163          JOptionPane.showMessageDialog(null, "Menu de dados!");
164      }
165
166      //metodo para exibir a mensagem de opção
167      public void exibirMensagemOpcao() {
168          JOptionPane.showMessageDialog(null, "Opção de dados!");
169      }
170
171      //metodo para exibir a mensagem de seleção
172      public void exibirMensagemSelecao() {
173          JOptionPane.showMessageDialog(null, "Seleção de dados!");
174      }
175
176      //metodo para exibir a mensagem de filtro
177      public void exibirMensagemFiltro() {
178          JOptionPane.showMessageDialog(null, "Filtro de dados!");
179      }
180
181      //metodo para exibir a mensagem de ordenação
182      public void exibirMensagemOrdenacao() {
183          JOptionPane.showMessageDialog(null, "Ordenação de dados!");
184      }
185
186      //metodo para exibir a mensagem de paginação
187      public void exibirMensagemPaginacao() {
188          JOptionPane.showMessageDialog(null, "Paginação de dados!");
189      }
190
191      //metodo para exibir a mensagem de exportação
192      public void exibirMensagemExportacao() {
193          JOptionPane.showMessageDialog(null, "Exportação de dados!");
194      }
195
196      //metodo para exibir a mensagem de importação
197      public void exibirMensagemImportacao() {
198          JOptionPane.showMessageDialog(null, "Importação de dados!");
199      }
200
201      //metodo para exibir a mensagem de backup
202      public void exibirMensagemBackup() {
203          JOptionPane.showMessageDialog(null, "Backup de dados!");
204      }
205
206      //metodo para exibir a mensagem de restauração
207      public void exibirMensagemRestauracao() {
208          JOptionPane.showMessageDialog(null, "Restauração de dados!");
209      }
210
211      //metodo para exibir a mensagem de cópia
212      public void exibirMensagemCopia() {
213          JOptionPane.showMessageDialog(null, "Cópia de dados!");
214      }
215
216      //metodo para exibir a mensagem de colagem
217      public void exibirMensagemColagem() {
218          JOptionPane.showMessageDialog(null, "Colagem de dados!");
219      }
220
221      //metodo para exibir a mensagem de corte
222      public void exibirMensagemCorte() {
223          JOptionPane.showMessageDialog(null, "Corte de dados!");
224      }
225
226      //metodo para exibir a mensagem de cola
227      public void exibirMensagemCola() {
228          JOptionPane.showMessageDialog(null, "Cola de dados!");
229      }
230
231      //metodo para exibir a mensagem de desfazer
232      public void exibirMensagemDesfazer() {
233          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
234      }
235
236      //metodo para exibir a mensagem de desfazer
237      public void exibirMensagemDesfazer() {
238          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
239      }
240
241      //metodo para exibir a mensagem de desfazer
242      public void exibirMensagemDesfazer() {
243          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
244      }
245
246      //metodo para exibir a mensagem de desfazer
247      public void exibirMensagemDesfazer() {
248          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
249      }
250
251      //metodo para exibir a mensagem de desfazer
252      public void exibirMensagemDesfazer() {
253          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
254      }
255
256      //metodo para exibir a mensagem de desfazer
257      public void exibirMensagemDesfazer() {
258          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
259      }
260
261      //metodo para exibir a mensagem de desfazer
262      public void exibirMensagemDesfazer() {
263          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
264      }
265
266      //metodo para exibir a mensagem de desfazer
267      public void exibirMensagemDesfazer() {
268          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
269      }
270
271      //metodo para exibir a mensagem de desfazer
272      public void exibirMensagemDesfazer() {
273          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
274      }
275
276      //metodo para exibir a mensagem de desfazer
277      public void exibirMensagemDesfazer() {
278          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
279      }
280
281      //metodo para exibir a mensagem de desfazer
282      public void exibirMensagemDesfazer() {
283          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
284      }
285
286      //metodo para exibir a mensagem de desfazer
287      public void exibirMensagemDesfazer() {
288          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
289      }
290
291      //metodo para exibir a mensagem de desfazer
292      public void exibirMensagemDesfazer() {
293          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
294      }
295
296      //metodo para exibir a mensagem de desfazer
297      public void exibirMensagemDesfazer() {
298          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
299      }
300
301      //metodo para exibir a mensagem de desfazer
302      public void exibirMensagemDesfazer() {
303          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
304      }
305
306      //metodo para exibir a mensagem de desfazer
307      public void exibirMensagemDesfazer() {
308          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
309      }
310
311      //metodo para exibir a mensagem de desfazer
312      public void exibirMensagemDesfazer() {
313          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
314      }
315
316      //metodo para exibir a mensagem de desfazer
317      public void exibirMensagemDesfazer() {
318          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
319      }
320
321      //metodo para exibir a mensagem de desfazer
322      public void exibirMensagemDesfazer() {
323          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
324      }
325
326      //metodo para exibir a mensagem de desfazer
327      public void exibirMensagemDesfazer() {
328          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
329      }
330
331      //metodo para exibir a mensagem de desfazer
332      public void exibirMensagemDesfazer() {
333          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
334      }
335
336      //metodo para exibir a mensagem de desfazer
337      public void exibirMensagemDesfazer() {
338          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
339      }
340
341      //metodo para exibir a mensagem de desfazer
342      public void exibirMensagemDesfazer() {
343          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
344      }
345
346      //metodo para exibir a mensagem de desfazer
347      public void exibirMensagemDesfazer() {
348          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
349      }
350
351      //metodo para exibir a mensagem de desfazer
352      public void exibirMensagemDesfazer() {
353          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
354      }
355
356      //metodo para exibir a mensagem de desfazer
357      public void exibirMensagemDesfazer() {
358          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
359      }
360
361      //metodo para exibir a mensagem de desfazer
362      public void exibirMensagemDesfazer() {
363          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
364      }
365
366      //metodo para exibir a mensagem de desfazer
367      public void exibirMensagemDesfazer() {
368          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
369      }
370
371      //metodo para exibir a mensagem de desfazer
372      public void exibirMensagemDesfazer() {
373          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
374      }
375
376      //metodo para exibir a mensagem de desfazer
377      public void exibirMensagemDesfazer() {
378          JOptionPane.showMessageDialog(null, "Desfazer de dados!");
379      }
380
381      //metodo para exibir a mensagem de desfazer
382      public void exibirMensagemDesfazer() {
383          JOptionPane.showMessageDialog(null, "Desfazer
```

```
Connecting to VM Service at http://127.0.0.1:55795/JewaYcMqrX0-/
Início da Aplicação. Chamando Função Lenta.
0
Operação lenta de 5 segundos.
5006
Continuando a Main.
5006
Executado operação rápida de 1 segundo.
6012
Executado outra operação rápida de 1 segundo.
7016
Executado operação lenta de 7 segundos.
14025
Final da Main.
14025
Exited
```

- **Vejamos agora o mesmo exemplo colocando assincronia e futuro no código**

```

class Teste {
    private String nome;

    public Teste(String nome) {
        this.nome = nome;
    }

    public void imprimir() {
        System.out.println("Nome: " + nome);
    }
}

public class Main {
    public static void main(String[] args) {
        Teste teste = new Teste("Fortale");
        teste.imprimir();
    }
}

```



```
Connecting to VM Service at http://127.0.0.1:64372/ETCxxU10bsw=/
Início da Aplicação. Chamando Função Lenta.
0
5
Continuando a Main.
5
Executado operação rápida de 1 segundo.
1015
Executado outra operação rápida de 1 segundo.
2029
Executado operação lenta de 7 segundos.
9040
Final da Main.
9041
Operação lenta de 5 segundos.
Exited
```

Fonte: Sato, 2022.

Assincronia

- Permite que uma operação lenta rode em paralelo enquanto o código faz outras tarefas
- Ex.: baixar imagens da internet, conectar a um servidor, etc.